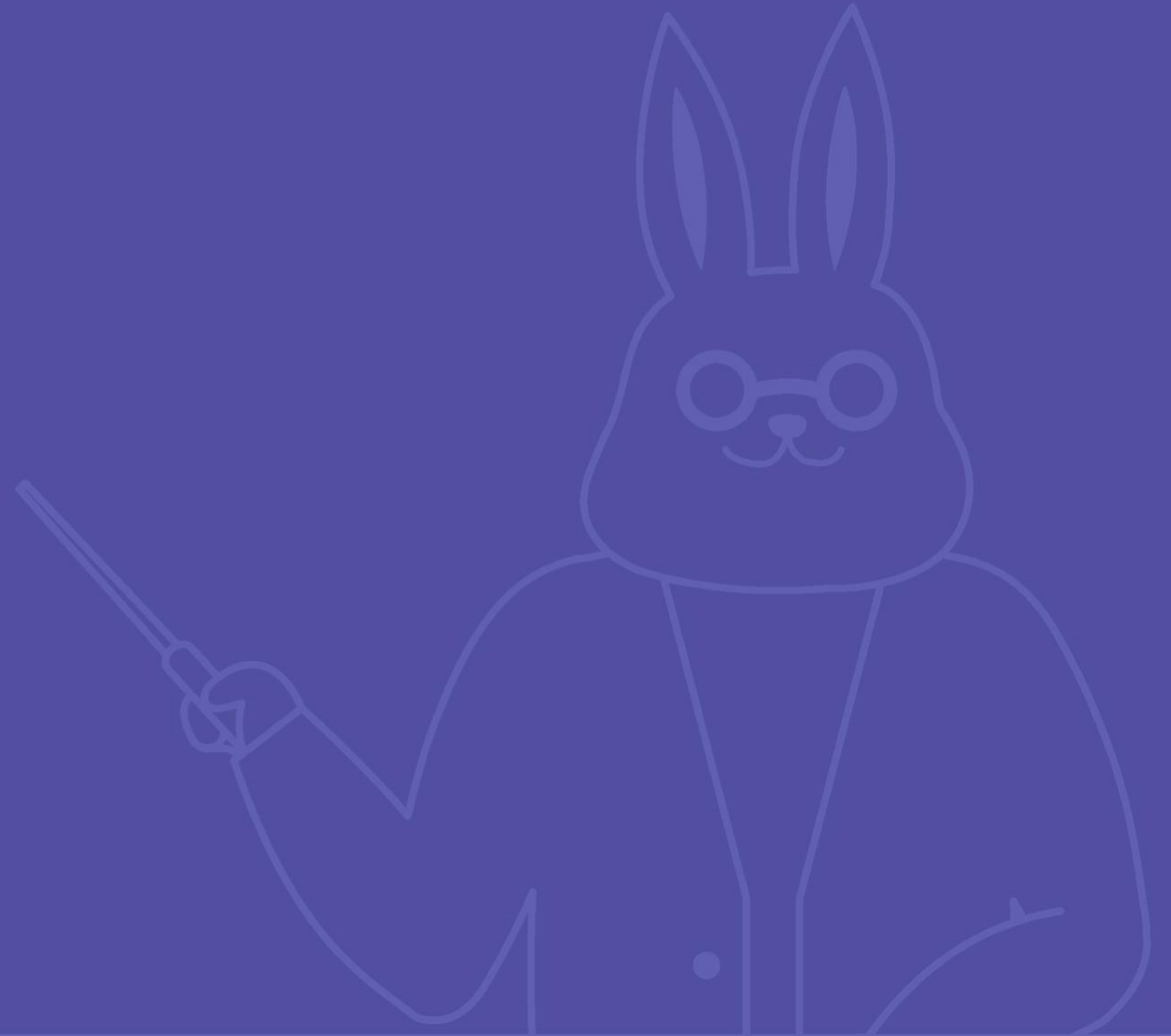




Flask 기초

05 REST API와 테스트



목차

01. REST API

02. flask-restful

03. Ajax란 – 동기/비동기

04. 테스트

수강목표

Flask를 사용한 REST API를 만들 수 있다.

개념을 간단히 배웠던 REST API에 대해서 다시 짚어보고, REST API를 구현 할 수 있습니다.

비동기 통신을 이해하고 사용 할 수 있다.

Ajax를 사용한 비동기통신을 이해하고 사용할 수 있습니다.

API 테스트 툴을 이해하고 사용 할 수 있다.

테스팅 코드를 작성해서 내가 만든 API를 테스트 할 수 있습니다.

01

REST API



✓ REST API란?

REST는 자원을 이름으로 구분하여 **해당 자원의 상태를 주고받는 모든 것**을 의미합니다. 한마디로 정리하면, **자원(resource)의 표현(representation)에 의한 상태 전달**입니다.

✓ REST의 구성과 특징

자원 - **URI**에 표현이 되어야 함 → 무엇을 서버에 요청할 것인지

행위 - **HTTP Method** → 어떻게(어떤 방법을) 요청할 것인지

표현 - **Representations** → **API만 보고 무엇을 요청할 것인지 알 수 있도록**

REST 서버는 API 제공, 클라이언트는 사용자 인증 등을 직접 관리하는 구조로 각각의 역할이 확실히 구분되기 때문에 클라이언트와 서버에서 개발해야 할 내용이 명확해지고 서로 간 의존성이 줄어들게 됩니다.

REST는 상태정보를 따로 저장하고 관리하지 않습니다. 세션 정보나 쿠키 정보를 별도로 저장하고 관리하지 않기 때문에 API 서버는 들어오는 요청만을 단순히 처리하면 됩니다.

서비스의 자유도가 높아지고 서버에서 불필요한 정보를 관리하지 않음으로써 구현이 단순해집니다.

✓ REST API 디자인 가이드

1. URI는 자원을 표현해야 함

REST API로 요청할 때에, URI에서 어떤 데이터를 요청하는지 표현이 되어야 합니다.

2. 자원에 대한 행위는 HTTP Method로 나타내야 함

자원에 대한 동작을 꼭 표현해야 합니다.

REST API에서 동작을 표현하는 방법은 HTTP Method입니다.

GET	조회 - GET을 통해 리소스를 조회합니다.
POST	생성 - 요청된 리소스를 생성합니다.
PUT	수정 - 리소스를 업데이트(수정)합니다.
DELETE	삭제 - DELETE를 통해 리소스를 삭제합니다.

✓ REST API 디자인 예시

1. 리소스 요청은 주로 동사보다는 명사를 사용

GET /members/delete/1	GET /members/show/1
DELETE /members/1	GET /members/1

GET /members/insert/10
POST /members/10

2. 슬래시(/)는 계층 관계를 나타냄

<http://elice.example.co.kr/lecture/python>
<http://elice.example.co.kr/user/students>

✓ REST API 디자인 예시

3. 파일 확장자는 URI에 포함하지 않음

`http://elice.example.co.kr/lecture/image/1.jpg`

`GET elice.example.co.kr/lecture/image/1`

4. 긴 URI에서는 밑줄(_)보다는 하이픈(-)을 사용

`http://elice.example.co.kr/board/free-talk`

✔ HTTP 응답 상태 코드

상태 코드	내용
200	정상적으로 수행
201	성공적으로 리소스 생성
400	클라이언트의 요청이 부적절할 때
401	인증되지 않은 상태에서 리소스 요청
404	응답하고 싶지 않은 리소스, 혹은 없는 리소스를 요청
500	서버에 문제가 생겼을 때

02

flask-restful



✓ flask-restful

Flask는 return 값을 jsonify로 주어서 RESTful API를 만들 수 있습니다.
하지만, 좀 더 **RESTful에 맞게 서버를 만들 수 있는 라이브러리**가 있습니다.
flask-restful을 활용해서 REST API 서버를 만들어 보겠습니다.

✓ 코드 비교

flask

```
@app.route('/first', methods=['GET'])
def elice_route():
    return jsonify('elice GET')

@app.route('/first', methods=['POST'])
def elice_route():
    return jsonify('elice POST')
```

restful

```
class First(Resource):
    def get(self):
        return 'elice GET', 200

    def post(self):
        return 'elice POST', 200
```

03

Ajax란 - 동기/비동기

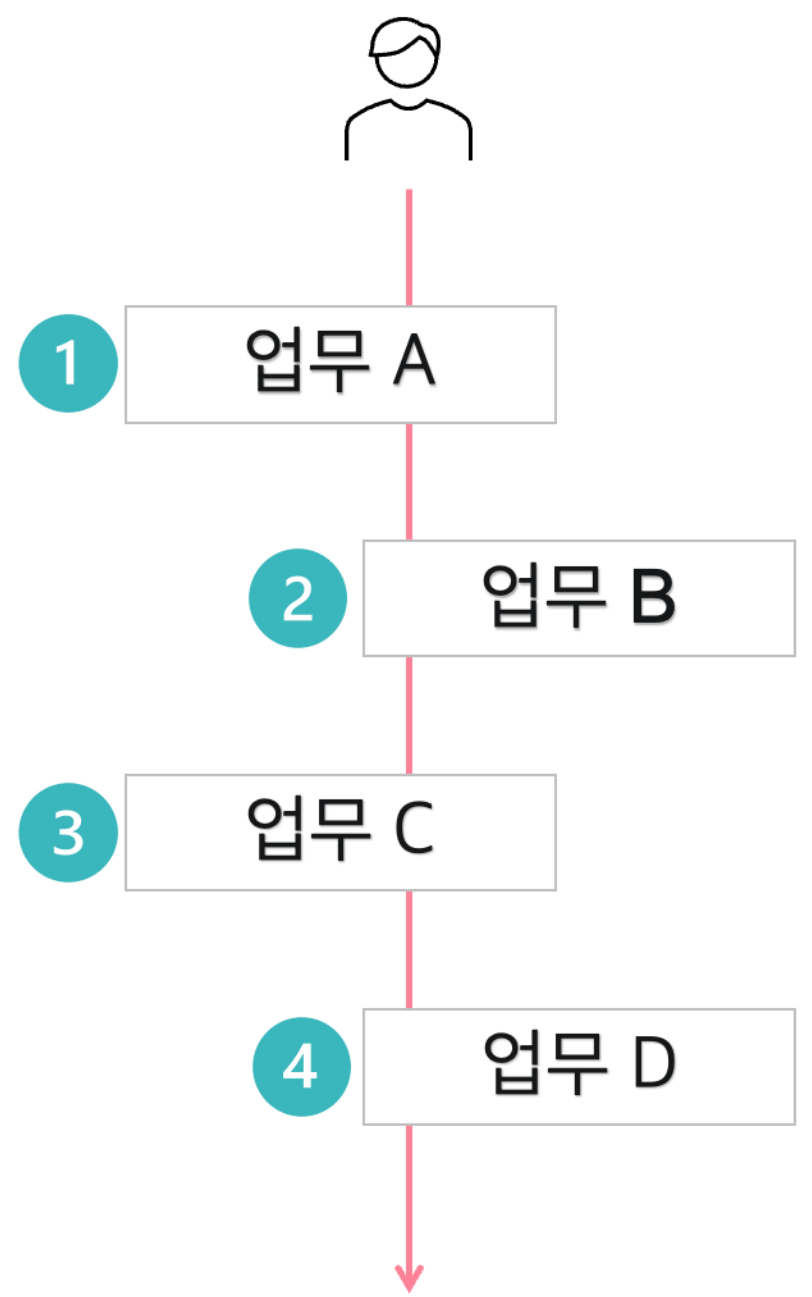


✓ 동기? 비동기?

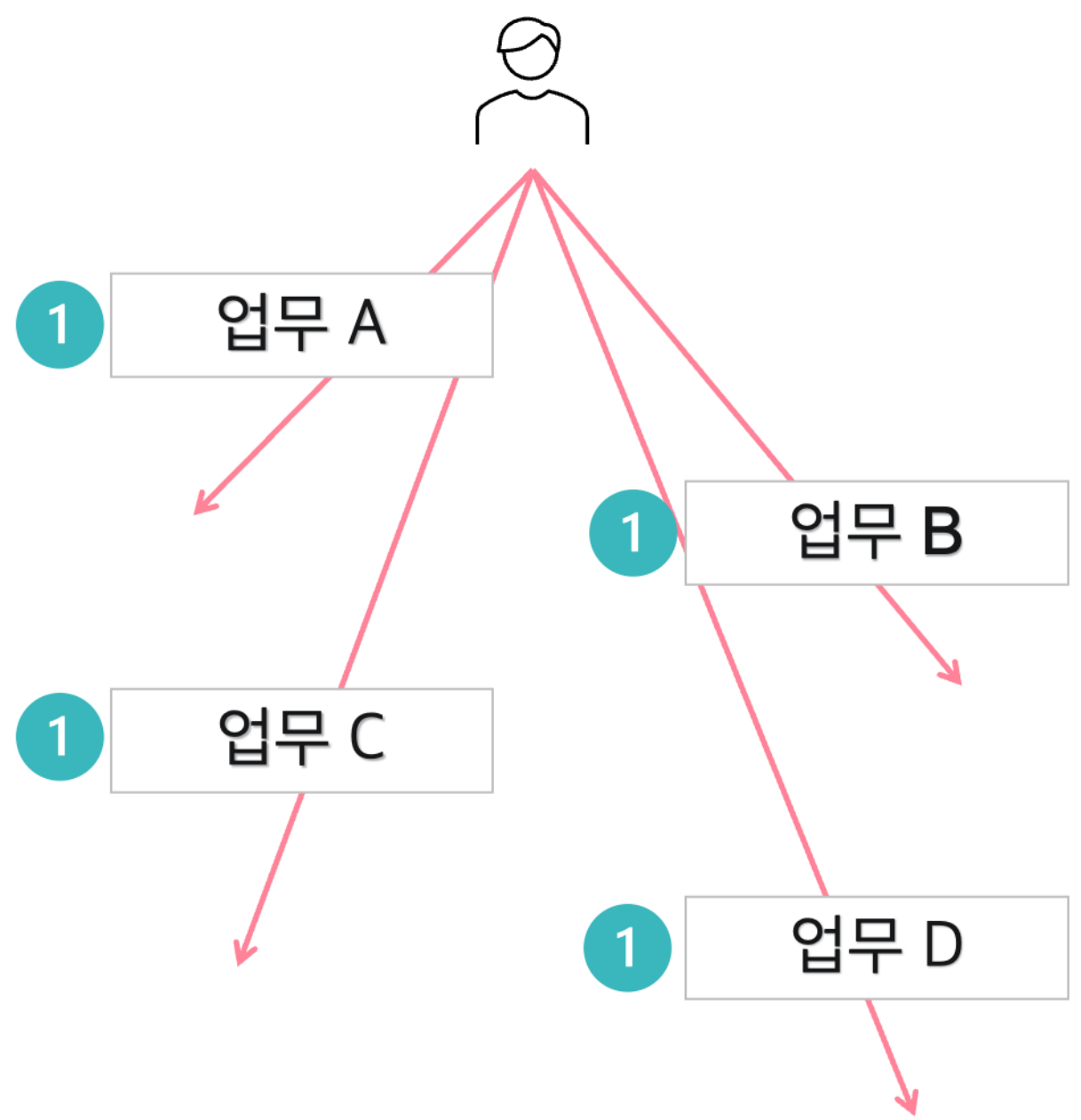
동기 – 앞의 작업이 끝나지 않는다면 다음 작업을 할 수 없다.

비동기 – 앞의 작업 상태와 상관없이 다음 동작을 수행할 수 있다.

✓ 동기와 비동기



동기



비동기

✓ 비동기 처리

Fetch

AXIOS

AJAX

✓ Ajax의 형태

예시

```
$.ajax({  
  url: 'URL주소',  
  type: '메서드',  
  data: {},  
  success : function(res){  
    console.log(res)  
  }  
})
```

- url은 Ajax로 요청을 보낼 주소를 적는 곳
- type은 Ajax로 보낼 방법을 적는 곳
- data는 Ajax 통신으로 보낼 데이터를 적는 곳
- success는 통신이 성공한 후 실행되는 함수를 적는 곳

✓ Ajax 호출 예시

예시

API doc 주소 : <http://data.seoul.go.kr/dataList/OA-394/S/1/datasetView.do>

```
function ajaxTest(){
    $.ajax({
        url: 'http://openapi.seoul.go.kr:8088/5a4947666d62613936345971506a6d/json/SearchParkInfoService/1/10',
        type: 'get',
        data: {},
        success : function(res){
            console.log(res)
        }
    })
}

///  

//HTML  

<button onclick = ajaxTest()> 호출 </button>
```

✓ Ajax – Flask

html

```
$.ajax({  
  url: 'ajax_test',  
  type: 'GET',  
  data: {},  
  success : function(res){  
    console.log(res)  
    //{ "name" : "elice"}  
  }  
})
```

flask

```
from flask import Flask, jsonify  
app = Flask(__name__)  
  
@app.route("/ajax_test")  
def elice_jsonify():  
    dict_data = {  
        "name" : "elice"  
    }  
    return jsonify(dict_data)
```

04

Flask Test



✓ 테스트

테스트 코드는 내가 작성한 기능들이 의도대로 잘 수행하는지
기능 확인을 위한 **테스트**를 위해 작성합니다.

✓ 테스트의 장점

테스트 환경 세팅 자동화

특수한 상황에 맞는 파라미터도 사전에 정의할 수 있기 때문에, 매번 파라미터 값을 수동으로 바꿔가면서 테스트하지 않아도 됩니다.

통합 테스트 시간을 줄임

통합 테스트에는 테스트코드로 검증하기 어려운 부분(클라이언트 인터페이스 / 배치 스크립트와의 호환성 등)에만 집중할 수 있습니다.

✓ 테스트의 장점

외부와 의존성 있는 로직을 테스트하기 편리

외부와 통신하는 로직을 Mock으로 처리해두면, 외부에서 발생할 수 있는 여러 환경을 내가 가짜로 구성할 수 있습니다.

전체 테스트 자동화

모든 세부 기능을 통합 테스트에서 다 확인하기는 어렵다. 전체 테스트가 자동화되면, 수정된 코드가 다른 기능에 미친 영향을 쉽게 확인할 수 있습니다.

✓ GET 테스트

flask

```
@app.route('/mysum', methods=['GET'])
def add():
    data = request.args
    num1 = int(data.get("a"))
    num2 = int(data.get("b"))
    return jsonify(
        {'result': num1 + num2 }
    )
```

test

```
def test_get():
    response = app.test_client().get(
        '/mysum?a=10&b=20',
    )
    data = json.loads(
        response.get_data()
    )
    assert response.status_code == 200
    assert data['result'] == 200
```

✓ POST 테스트

flask

```
@app.route('/add', methods=['POST'])
def add():
    data = request.get_json()
    return jsonify(
        {'sum': data['a'] + data['b']}
    )
```

test

```
def test_add():
    response = app.test_client().post(
        '/add',
        data=json.dumps({'a': 10, 'b': 2}),
        content_type='application/json',
    )

    data = json.loads(
        response.get_data(as_text=True)
    )
    assert response.status_code == 200
    assert data['sum'] == 12
```

크레딧

/* elice */

코스 매니저

이재성

콘텐츠 제작자

이바울, 이재성

강사

이바울

감수자

이바울

디자이너

강혜정

연락처

TEL

070-4633-2015

WEB

<https://elice.io>

E-MAIL

contact@elice.io

